

# 1. Levantamento de Requisitos

Requisitos Funcionais Prioritários

**RF01 – Submissão de Ticket:** O utilizador deve conseguir submeter um formulário com Título, Descrição, Categoria (TI, Manutenção, Compras), Prioridade (Baixa, Média, Alta, Urgente) e Localização (Ermesinde, Madeira, Tânger).

**RF02 – Listagem de Tickets:** O sistema deve apresentar uma lista com todos os tickets submetidos.

**RF03 – Atualização de Tickets:** O sistema deve permitir a atualização de Estado (novo, em curso, resolvido) dos tickets.

**RF04 – Filtragem de Tickets:** O sistema deve permitir filtrar os tickets submetidos por Categoria, Prioridade, Localização e Estado.

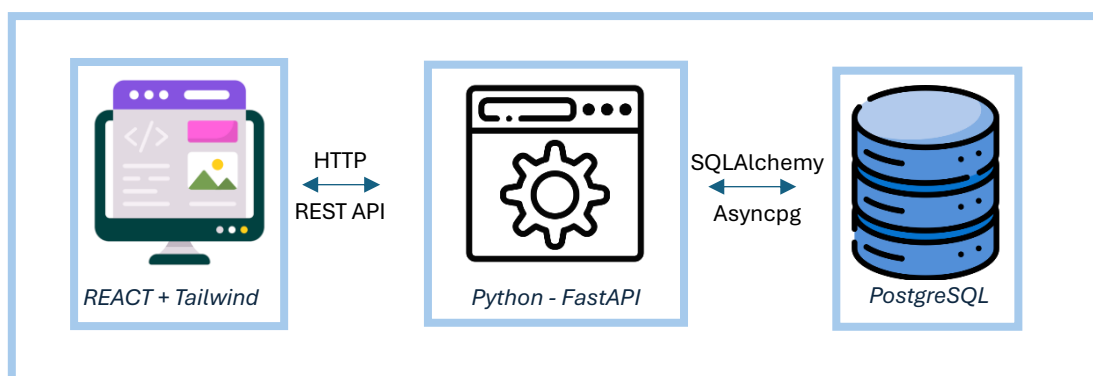
Requisitos Não-Funcionais Prioritários:

**RNF01 – Simplicidade e Usabilidade:** O sistema deve ter uma interface intuitiva e limpa, pensada para utilizadores não técnicos de qualquer geografia do grupo.

**RNF02 – Persistência de Dados:** O sistema deve garantir que os dados não se perdem ao reiniciar o servidor.

**RNF03 – Separação de Responsabilidades:** Arquitetura desacoplada e modular garantindo uma fácil manutenção futura.

## 2. Arquitetura, Stack e Decisões de Design



### 1. Frontend

- a. React – Escolhido pela facilidade na criação de uma Single Page Application.
- b. Tailwind – Facilidade na criação de estilos in-line garantindo muita variabilidade nessa criação (ao contrário do bootstrap que nos oferece componentes pre-styled)

## 2. Backend

- a. Python com FastAPI – Escolhido essencialmente pela facilidade na transição para o tratamento de dados que as libraries de data science / machine learning em python oferecem como scikit, numpy, polars, pandas, tensorflow, pytorch, etc.

## 3. Base de dados

- a. PostgreSQL – Escolhido por ser um gold standard na indústria e por ser relacional.

## 4. Arquitetura global

- a. Docker – Escolhido por garantir reprodutibilidade local e remota, isolando cada container.

## UML - Base de Dados

| <b>Ticket</b>           |
|-------------------------|
| -id: UUID               |
| -title: string          |
| -description: text      |
| -category: CategoryType |
| -priority: PriorityType |
| -location: LocationType |
| -status: StatusType     |
| -created_at: dateTime   |
| -updated_at: dateTime   |

| <<enumeration>><br>CategoryType |
|---------------------------------|
| TI                              |
| Maintenance                     |
| Purchases                       |
| Finances                        |
| Customer_Service                |
| Production                      |

| <<enumeration>><br>PriorityType |
|---------------------------------|
| Low                             |
| Medium                          |
| High                            |
| Urgent                          |

| <<enumeration>><br>LocationType |
|---------------------------------|
| Ermesinde                       |
| Madeira                         |
| Tânger                          |

| <<enumeration>><br>StatusType |
|-------------------------------|
| New                           |
| In_Progress                   |
| Closed                        |

## 3. Uso de IA

O co-pilot do VSCode foi utilizado para code suggestions. O Gemini foi utilizado para um screening inicial do projeto e também questionado que caminhos poderiam ser tomados garantindo uma reflexão dos padrões de indústria adaptando ao contexto do problema. Por exemplo, o Gemini recomendou utilizar

Javascript (Next.js) para ambos frontend e backend; SQLite ou PostgreSQL para a base de dados e utilizar Vercel para o deployment visto a sua integração com o Github. No entanto, sabendo a minha preferência por Python, o conhecimento de trabalhar com React e também a quantidade de dados que este sistema pode gerar, a preferência da utilização deste stack fez-me mais sentido (além das razões acima descritas). A IA também não sugeriu a utilização de Docker, embora seja uma ferramenta que acho fundamental existir especialmente em trabalhos de equipa.

Além de alguns pedidos de tentar entender melhor a forma como fui escrevendo o código e das bibliotecas utilizadas e a resolução de alguns bugs ao longo do caminho, utilizei o co-pilot do VSCode para a escrita do README.md, das docstrings do frontend e do backend e da criação dos módulos de logs ao longo da codebase.

Ou seja, a minha utilização de IA passa muito por perceber que alternativas existem e saber os seus pontos fortes e fracos (fazendo parecer uma análise SWOT), utilização da sua rapidez para tarefas repetitivas e longas como escrita de documentos ou módulos simples.

## 4. Próximos Passos

- Implementação de testes unitários para o backend e frontend.
- Autenticação e Hosting do frontend através do Firebase
- Utilização de HTTPS em vez de HTTP para garantir segurança na comunicação
- Update da base de dados com tabela de users e relações respetivas
- Deployment do backend em algum serviço cloud serverless
- Deployment da base de dados em algum PaaS escalável, alta disponibilidade e baixo custo
- Utilização de Github Actions para CI/CD

## 5. Integração com ERPs

Não conhecendo os softwares, mas assumindo que estes garantem uma comunicação externa através de APIS poderemos ter duas soluções:

1. Utilização dos dados gerados pelo sistema de tickets para visualização duplicada tanto no frontend do sistema como também nos ERPs, o que

garante uma visualização fácil para outros stakeholders que não utilizam o sistema de tickets

2. Remoção por completo do frontend do sistema de tickets e utilização, se possível, do sistema dentro dos ERPs através de, novamente, APIs. Isto garante a utilização de apenas 1 ferramenta por parte dos stakeholders.
3. Combinação entre os dois sistemas para complementarem as suas funções. Isto é, por exemplo, quando se cria um pedido para compras de algum item, é ativado um endpoint nos ERPs que irão colocar a ordem do produto em andamento.